

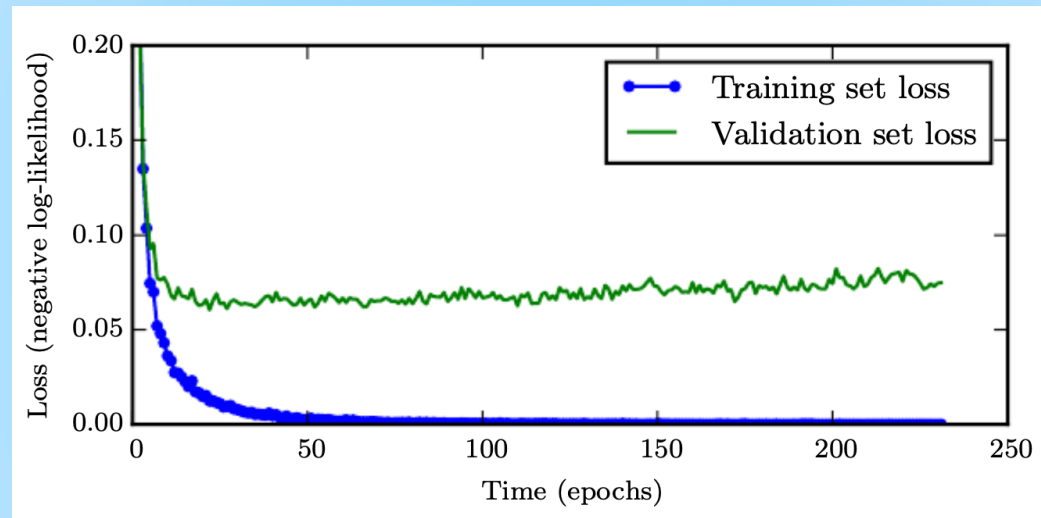


Deep Learning

Dr. Shay Maymon

Early Stopping: A Simple Regularization Strategy

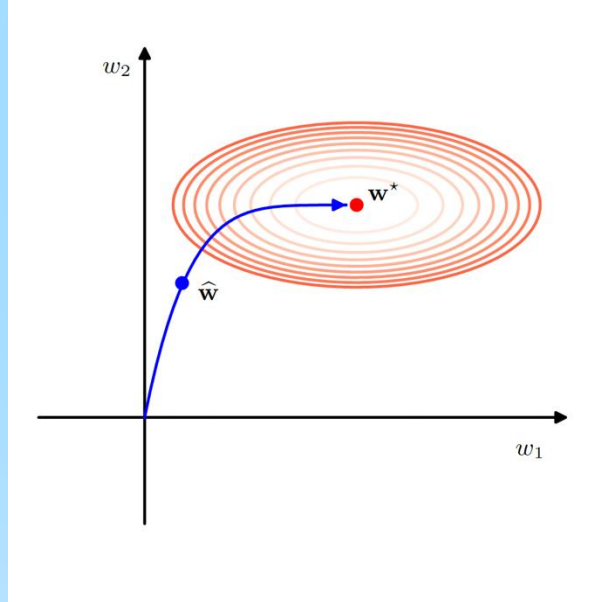
Stop training when validation performance stops improving



Early stopping acts as a regularizer

- Early stopping has the effect of restricting the optimization procedure to a relatively small volume of parameter space in the neighborhood of the initial parameter value

$$\tau \approx \frac{1}{\varepsilon \alpha}, \alpha \approx \frac{1}{\varepsilon \tau}$$



Parameters that correspond to **directions of significant curvature** tend to **learn early** relative to parameters corresponding to **directions of less curvature**

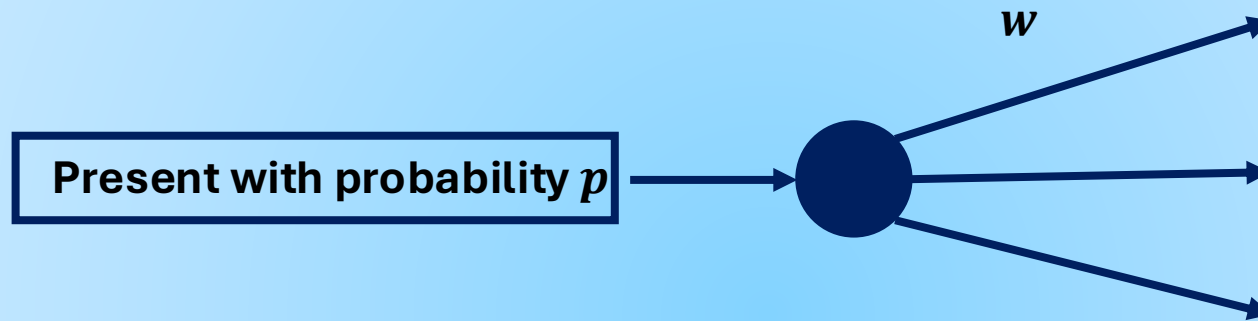
Ensemble Methods

- Train multiple models independently
- Combine outputs at test time:
 - ✓ Classification: use **majority vote**
 - ✓ Regression: **average** predictions
- Benefit:
 - ✓ Reduces generalization error by combining several models



Dropout: Random Unit Selection

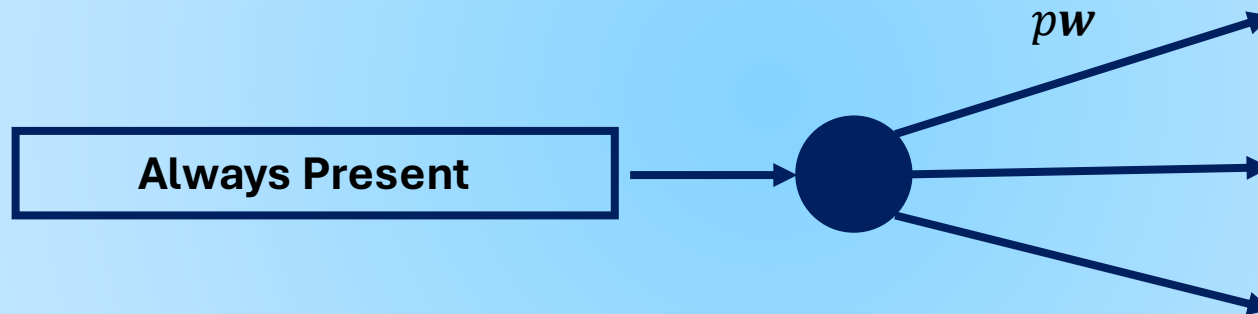
- Each unit is retained randomly with probability p



- p can be tuned using a validation set
- Dropping out 20% of the input units and 50% of the hidden units was often found to be optimal

Dropout at Test Time

- **Averaging** over exponentially many subnetworks is **infeasible**
- Instead, use a **single deterministic network** with:
 - ✓ No dropout
 - ✓ Scaled weights



- **Weight Scaling Rule:**
 - ✓ If a unit was kept with probability p during training multiply its outgoing weights by p at test time

Toward Faster Dropout: Marginalizing the Noise

A decorative graphic in the top right corner of the slide, featuring a network of blue nodes connected by thin, glowing blue lines, resembling a simplified neural network or a data flow diagram.

- Dropout = **stochastic regularization** using random noise (unit dropping). **Injecting noise** into hidden unit activations
- In dropout, we **minimize the loss** function **stochastically** under a noise distribution
- This **can be seen** as **minimizing an expected loss** function

Deterministic Interpretation of Dropout

- Idea:
 - ✓ Get dropout's benefits **without stochasticity**
 - ✓ **Marginalize the noise** to derive an **equivalent deterministic regularizer**
 - ✓ These models:
 - ❖ Approximate dropout **without sampling**
 - ❖ **Require no random bits** during **training** or **inference**
 - ❖ **Allow exact gradients** of the marginalized loss function
 - ❖ In **simple** cases, dropout can be **analytically marginalized** out to obtain **deterministic regularization methods**
 - ❖ For **complex** models, deriving this regularizer is **non-trivial**

Dropout in Linear Regression

- **Linear Regression:**

$$\text{minimize}_w \|y - Xw\|^2$$

- **Dropout in Linear Regression:**

The input X is dropped out such that any input dimension is retained with probability p :

$$\tilde{X}_{i,j} = R_{i,j}X_{i,j} \text{ where } R_{i,j} \sim \text{Bernoulli}(p), \text{ iid}$$

$$E_R(\tilde{X}_{i,j}) = pX_{i,j}$$

$$E_R(\tilde{X}_{i,j}\tilde{X}_{k,l}) = (?)X_{i,j}X_{k,l}$$

Dropout in Linear Regression

- Linear Regression:

$$\text{minimize}_w \|y - Xw\|^2$$

- Dropout in Linear Regression:

The input X is dropped out such that any input dimension is retained with probability p :

$$\tilde{X}_{i,j} = R_{i,j}X_{i,j} \text{ where } R_{i,j} \sim^{iid} \text{Bernoulli}(p)$$

$$E_R(\tilde{X}_{i,j}) = pX_{i,j}$$
$$E_R(\tilde{X}_{i,j}\tilde{X}_{k,l}) = \begin{cases} p^2X_{i,j}X_{k,l} \\ pX_{i,j}^2 \end{cases}$$

Dropout in Linear Regression

$$\begin{aligned} & \text{minimize}_{\mathbf{w}} E_R \left(\|\mathbf{y} - \tilde{X}\mathbf{w}\|^2 \right) \\ & E_R \left(\|\mathbf{y} - \tilde{X}\mathbf{w}\|^2 \right) = \\ & E_R \left(\|\mathbf{y}\|^2 - 2\mathbf{y}^T \tilde{X}\mathbf{w} + \|\tilde{X}\mathbf{w}\|^2 \right) \\ & E_R \left(\|\mathbf{y}\|^2 - 2\mathbf{y}^T \tilde{X}\mathbf{w} + \mathbf{w}^T \tilde{X}^T \tilde{X} \mathbf{w} \right) = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T E_R(\tilde{X})\mathbf{w} + \mathbf{w}^T E_R(\tilde{X}^T \tilde{X})\mathbf{w} = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T (pX)\mathbf{w} + \mathbf{w}^T (p^2 X^T X + (p - p^2) \text{diag}(X^T X))\mathbf{w} = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T (pX)\mathbf{w} + \mathbf{w}^T ((pX)^T (pX))\mathbf{w} + p(1 - p)\mathbf{w}^T (\text{diag}(X^T X))\mathbf{w} = \\ & \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1 - p)\|\Gamma\mathbf{w}\|^2 \\ & \Gamma = (\text{diag}(X^T X))^{1/2} \end{aligned}$$

Dropout in Linear Regression

- In linear regression, dropout is equivalent in expectation to a modified form of L2 regularization

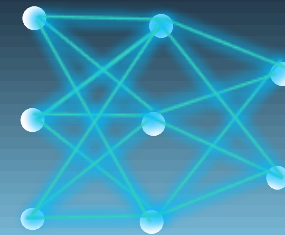
$$\text{minimize}_{\mathbf{w}} \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1 - p)\|\Gamma\mathbf{w}\|^2$$

$$\Gamma = (\text{diag}(X^T X))^{1/2}$$

Dropout: Summary and Impact



- **Problem:**
Standard training **encourages fragile co-adaptations** that don't generalize
- **Purpose:** **Reduce overfitting** and **improve generalization** in neural networks
- **Solution:**
 - ✓ Dropout randomly disables units during training
 - ✓ Forces units to work independently, improving robustness
- **Proven benefits across diverse domains**



Thank you